

NYU Spring 2024

Lecture 18: Input and Output or I/O

Professor G. Sandoval

18

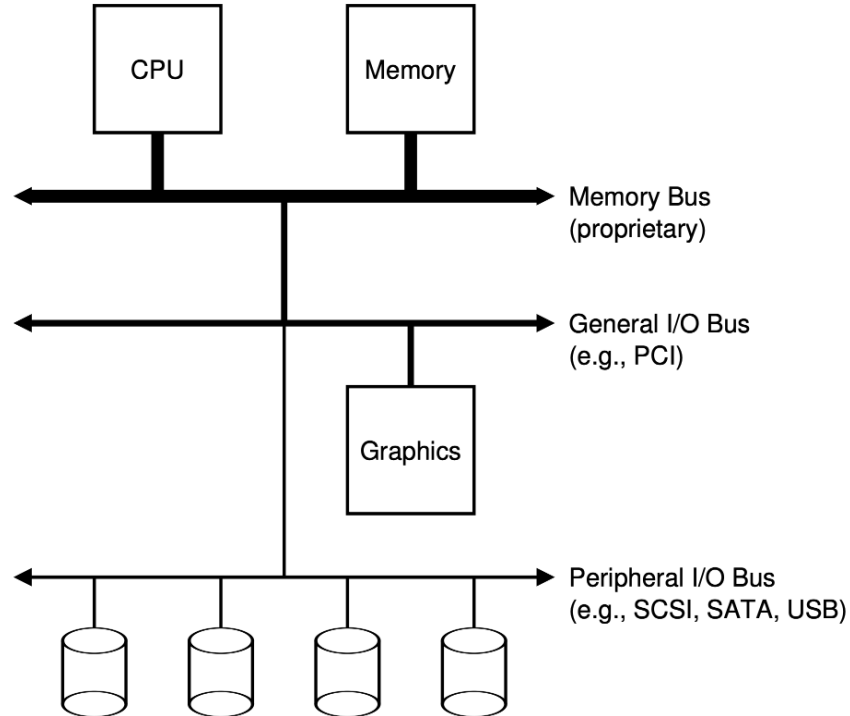


Some slides adapted by G. Sandoval for CS3224,
from slide by Brendan Dolan-Gavitt

Motivation

- Explain how a character appears on the screen. Use OS concepts like :
 - hardware interrupt
 - Interrupt Controller
 - Interrupt handler
 - Event Queue

Prototypical System Architecture



Drivers & I/O

- Code for handling interactions with hardware peripherals makes up a significant portion of most OSes
- Interacting with hardware is complicated and hard to get right
 - This is one reason we make OSes do it!
 - Another reason is that hardware is a shared resource we need to manage

Categories of I/O Devices

Can be grouped in 2 categories:

Block Devices

- Random access
- You can ask for any block of data with equal ease
- Eg: Disk Drives, SSD, floppy, CD

Character Devices

- Only support reading/writing streams
- Not structured into blocks
- Example: Serial Port

Device Controllers

- Hardware peripherals tend to have both mechanical and electronic component (the device controller)
- OS communicates with electronic component, which actually controls the mechanical bits

Driver vs Controller?

- **Device Drivers:** low level programs written by hardware manufacturers which are loaded as kernel modules and provide the kernel with the knowledge on how to control the devices
- **Device Controllers:** A device controller (hardware) understands "software" input. It translates software input into something a hardware device understands. Controllers sometimes have their own memory and their own CPU.

Device drivers are **purely software**, no matter where they are found. Device drivers communicate between the operating system and the physical device controller.

Device Controllers

- What does the electronic component do?
 - Provides a higher level interface to the actual mechanical operation of the device
 - Perform error correction on the low-level data retrieved before handing it back to the OS
 - Implement processing algorithms (such as DSP) in hardware, faster than the main CPU could

Firmware

- The device controllers also run software and have their own operating systems (*firmware*)
- It turns out that this is often upgradeable by the user (to fix bugs)

I/O Techniques

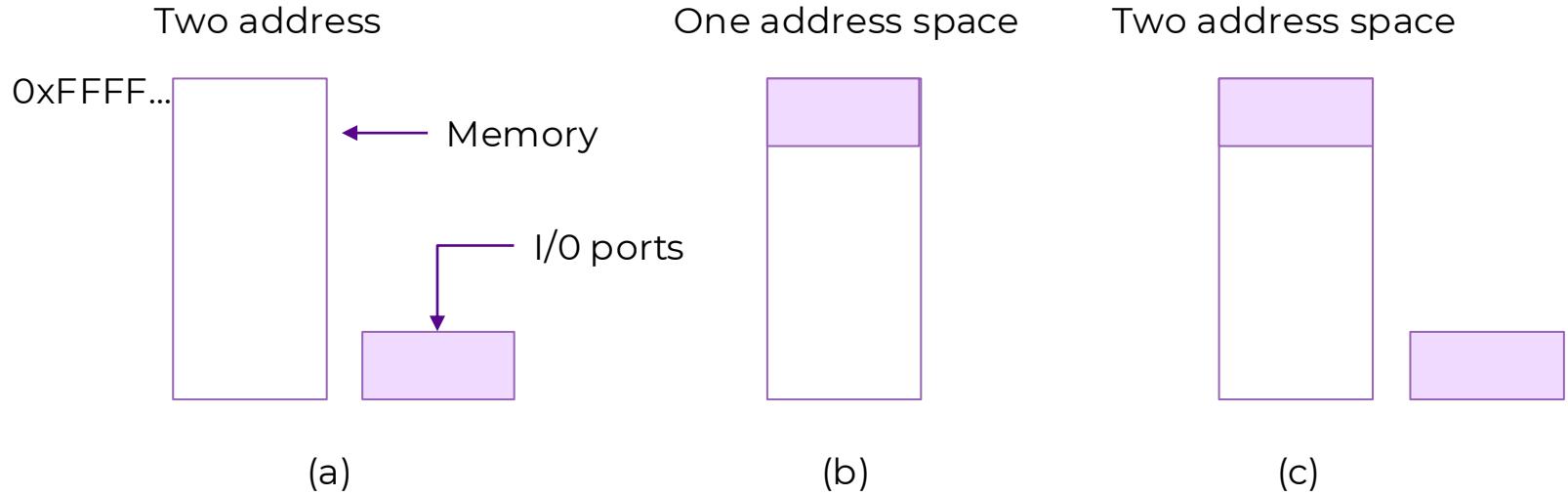
- Port I/O
- Memory Mapped I/O
- Direct Memory Access (DMA)

Port I/O

- There are a fixed number of *port addresses*
- Specialized CPU instructions (in and out) read and write bytes to these ports, which are connected to hardware devices:
 - `IN REG, PORT`
 - `OUT PORT, REG`
- Can be thought of as having a separate address space just for I/O

Memory Mapped I/O

- Contrast to port I/O: only one big address space
- Certain portions of physical memory are assigned to various hardware devices
- Reading from / writing to that address has the effect of receiving / sending data to the peripheral



- a. Port I/O
- b. Memory Mapped I/O
- c. Hybrid

Port I/O vs MMIO

- Port I/O
 - Need to have extra CPU instructions
 - Not accessible from C/C++ code (must use assembly)
- Memory mapped I/O
 - Sacrifice some physical address space
 - Can interact badly with memory caching
 - Each address must be checked to see if it is I/O

Direct Memory Access (DMA)

- Both port and memory-mapped I/O have a significant downside – require CPU to run for every piece of data transferred
- Instead, we can ask the device to do a bulk transfer to memory
- This transfer goes directly from the device to RAM, so the CPU can run concurrently

DMA Controller

- Communication between devices and RAM is mediated by the DMA controller
- CPU uses programmed I/O (Port I/O or MMIO) to configure the DMA controller first
 - What device to transfer from, where to transfer to, how much data
- DMA controller independently fetches bytes from the disk into main memory and then notifies the CPU when the transfer is complete

Direct Memory Access

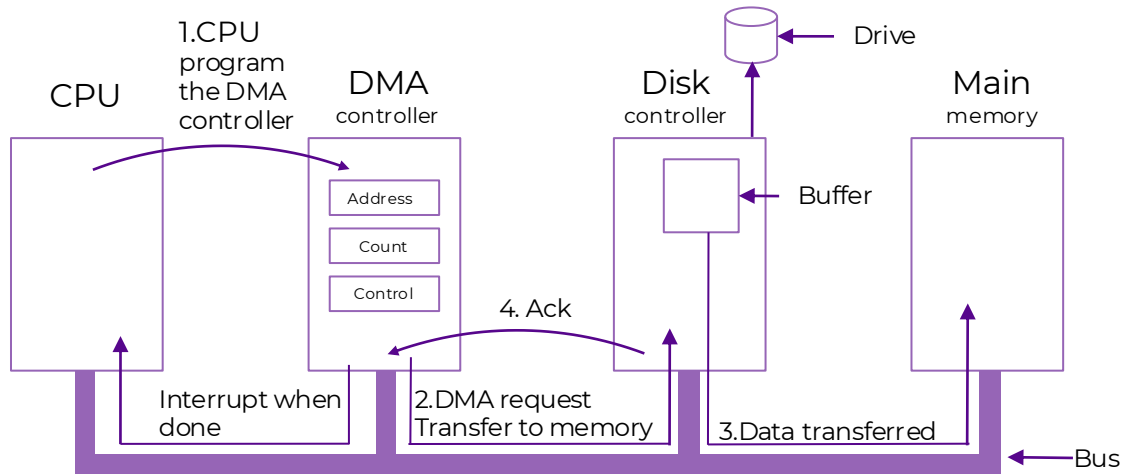


Figure 5-4. Operation of a DMA transfer.

Interrupts

- When hardware devices need attention for any reason (e.g., done with some work) they raise an *interrupt*
- Interrupts are implemented by asserting a signal on some bus line, which is detected by the *interrupt controller*
- Interrupt controller queues interrupts and delivers them to the CPU
- CPU handles each one and then tells the interrupt controller it has acknowledged the interrupt

Interrupt Handling

- When the CPU is notified of an interrupt, it stops whatever it's doing
 - At this point it needs to save some state so that it can come back to it later
- The interrupt will come with a number, which is used to index into a table (the *interrupt vector table*) to find the address of the handler
- The interrupt handler will acknowledge the interrupt, allowing the interrupt controller to deliver the next one

Interrupts Revisited

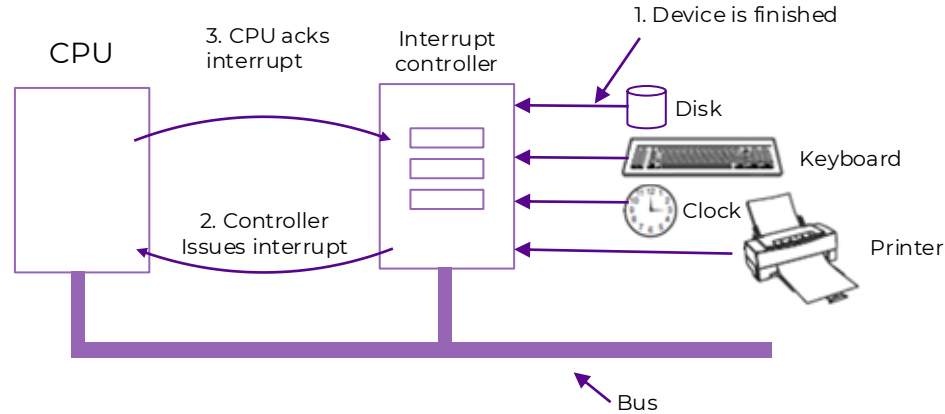


Figure 5-5. How an interrupt happens. The connections between the devices and the interrupt controller actually use interrupt lines on the bus rather than dedicated wires.

I/O – Software Side

- Make hardware I/O look simple and consistent:
 - Hide hardware differences – programs don't care which device is used.
 - Handle complexity internally – errors and edge cases managed by OS
 - Provide a simple interface – I/O appears predictable and (mostly) synchronous.
 - Manage sharing – multiple programs can safely use the same device

I/O Programming Model

- Three main kinds
 - Programmed I/O
 - Interrupt-driven I/O
 - DMA-based I/O

Programmed I/O

- CPU does all the work
- For each operation, poll the hardware, send or receive the data, and repeat until all data is sent or received
- Example: reading from a serial port

```
void ReadDebugChar(int *c) {  
    // Wait until serial port has data  
    while (ReadDword(0xbd370404) & 0x4 == 0) {  
        // busy loop  
    }  
    ReadDword(0xbd370414, &c);  
}
```

Interrupt-Driven I/O

- Instead of polling while waiting for hardware to be ready, we could ask the hardware to **tell us** via an **interrupt**
- Now we can go do other things while we wait for it to finish
- This can make a system much more **responsive** if the device is slow

DMA I/O

- This is essentially an extension of interrupt-driven I/O
- Instead of interrupting every time a piece of data is ready, program DMA controller for a bulk transfer
- Advantage over plain interrupt-driven access is that if your data is large, you get just one interrupt rather than many

Lect 18: Which I/O method wastes the MOST CPU time?

Programmed I/O (polling)

0%

Interrupt-driven I/O

0%

DMA

0%

None of the above

0%

```

    struct group_info *group_info)
{
    groupinfo->blocks[0] += group_info->small_
    int i;
    if (groupinfo->blocks[0] != group_info->small_
        for (i = 0; i < group_info->nblocks; i++)
            int i;
            freepage((unsigned long)groupinfo->blo
            for (i = 0; i < group_info->nblocks; i++)
            }
            freepage((unsigned long)groupinfo->blo
            kfree(groupinfo);
        }
        kfree(groupinfo);
    }
}
EXPORT_SYMBOL(groupsfree);

EXPORT_SYMBOL(groupsfree);
/* export the groupinfo to a user-space array */
static int groups_touser(gid_t_user *groupList,
/* export the groupinfo to a user-space array */
const struct group_info *group_info)
static int groups_touser(gid_t_user *groupList,
{
    const struct group_info *group_info)
    int i;
    unsigned int count = groupinfo->nblocks;
    int i;
    unsigned int count = groupinfo->nblocks;
    for (i = 0; i < group_info->nblocks; i++) {
        unsigned int cpcount = min(NGROUPSPERBLOCK
        for (i = 0; i < group_info->nblocks; i++) {
            unsigned int len = cpcount * sizeof(*group
            unsigned int cpcount = min(NGROUPSPERBLOCK
            cpcount * sizeof(*group

```

OS I/O Design Issues XV6 I/O Case Studies

I/O handling architecture

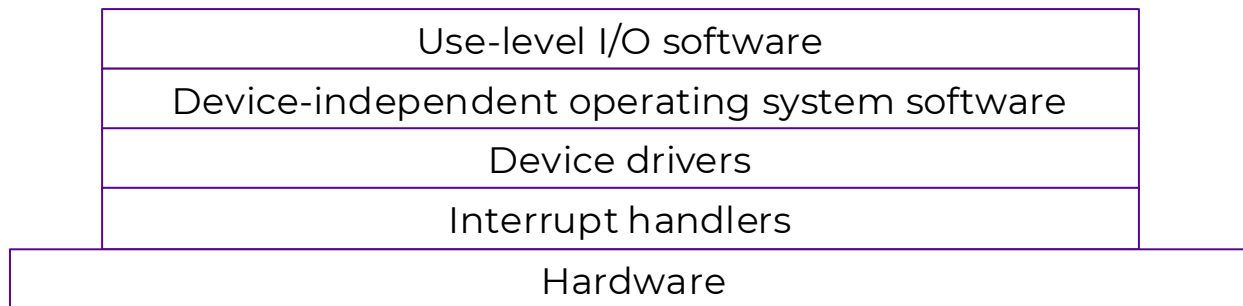


Figure 5-11. Layers of the I/O software system.

I/O handling architecture

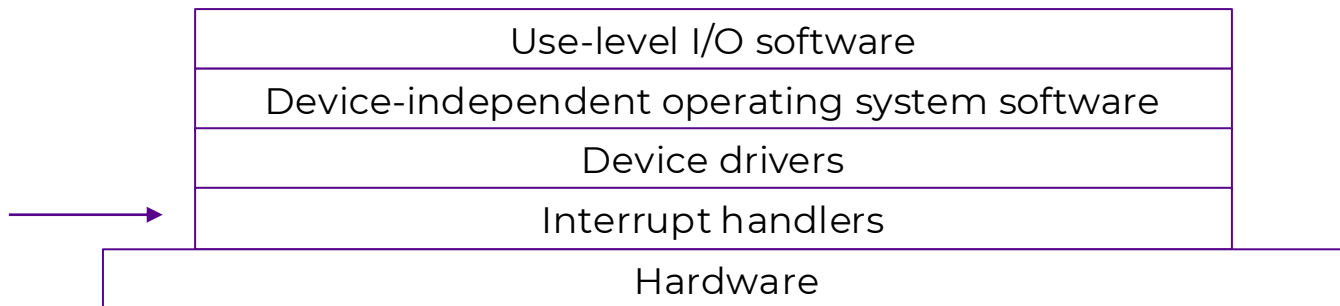


Figure 5-11. Layers of the I/O software system.

Tanenbaum & Bo, Modern Operating Systems: 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Interrupt Handlers

- Conceptually simple – just do what's necessary to handle the interrupt and then resume execution
- Reality is more complicated...

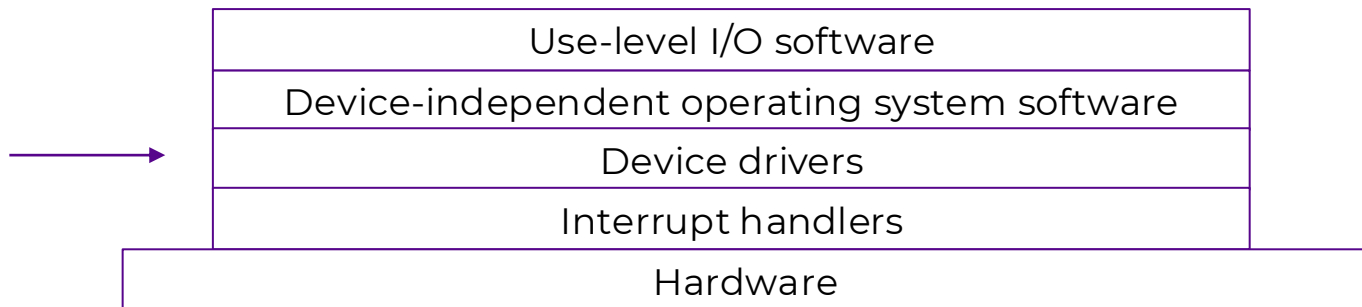
Interrupt Handlers

- Save additional registers
- Set up kernel context (e.g., load page tables or set TLB entries)
- Set up a stack for the handler
- Acknowledge the interrupt
- Run the interrupt handler
- Choose next process (may be able to wake up a blocked proc)
- Switch context to new process

Interrupt Handler Organization

- If the rate of interrupts is high and we take a while to service each one, we may fall behind
- To avoid this, interrupt handlers are often written to do the minimum possible work needed to acknowledge the interrupt
- They then queue the remaining work to do later (with interrupts enabled)
- In Linux, interrupt acknowledgement is called the *top half*, and the remaining work happens in the *bottom half*

I/O handling architecture



Device Drivers

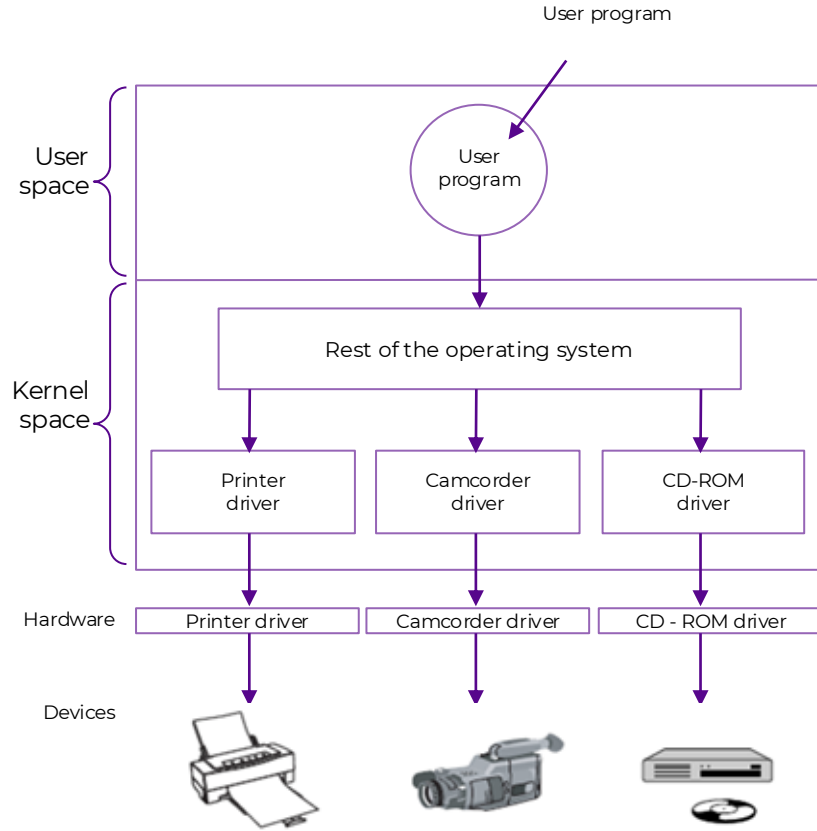


Figure 5-12. Logical positioning of device drivers. In reality all communication between drivers and device controllers goes over the bus.

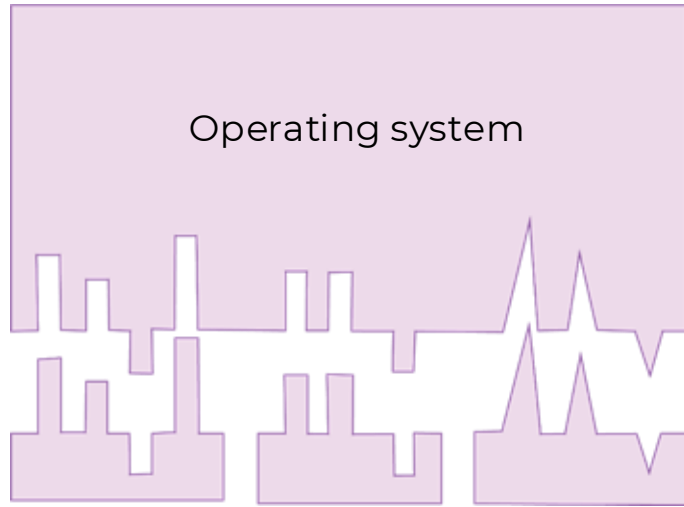
Device Drivers

- Device drivers do the actual work of interacting with the device controller
 - **Initialization** – talk to the controller to get device into a working state
 - **Control** – take commands from higher levels in OS and translate them into hardware requests
 - **Multiplexing** – check whether device is in use and defer until later if so
 - **Power management** – put the device to sleep when it's not being used

Driver APIs

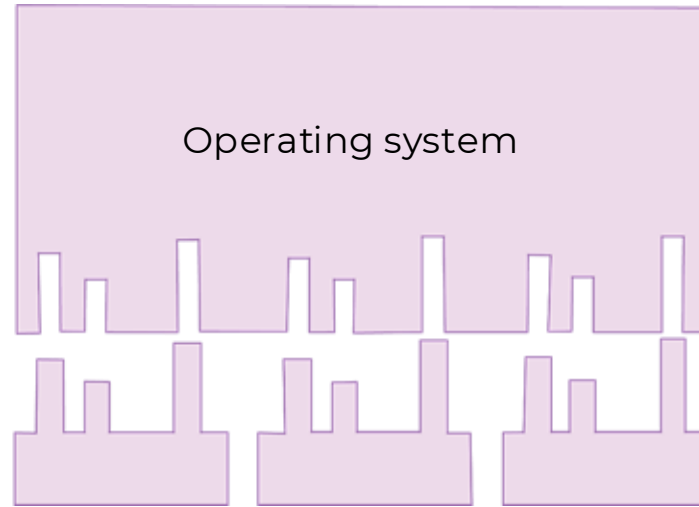
- Because it's convenient not to write drivers for every piece of hardware yourself, OS creators typically specify a *driver API*
- This API specifies a well-defined model for the OS to use when interacting with the driver, and says what kernel functions are available for driver use
- So if your OS is popular enough, hardware manufacturers will write drivers for you!

Uniform Interfacing for Device Drivers



SATA disk driver USB disk driver SCSI disk driver

(a)



SATA disk driver USB disk driver SCSI disk driver

(b)

Figure 5-14.

(a) Without a standard driver interface. (b) With a standard driver interface.

I/O handling architecture

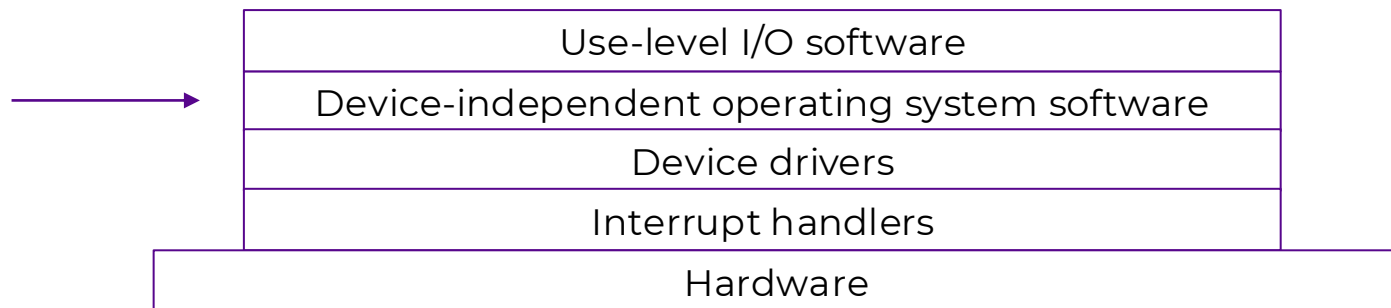


Figure 5-11. Layers of the I/O software system.

Generic I/O Layer

- Above the driver level, there may be a common layer for handling I/O issues common to multiple drivers
 - Buffering I/O requests
 - Generalized error reporting
 - Provide uniform block size

Buffering example

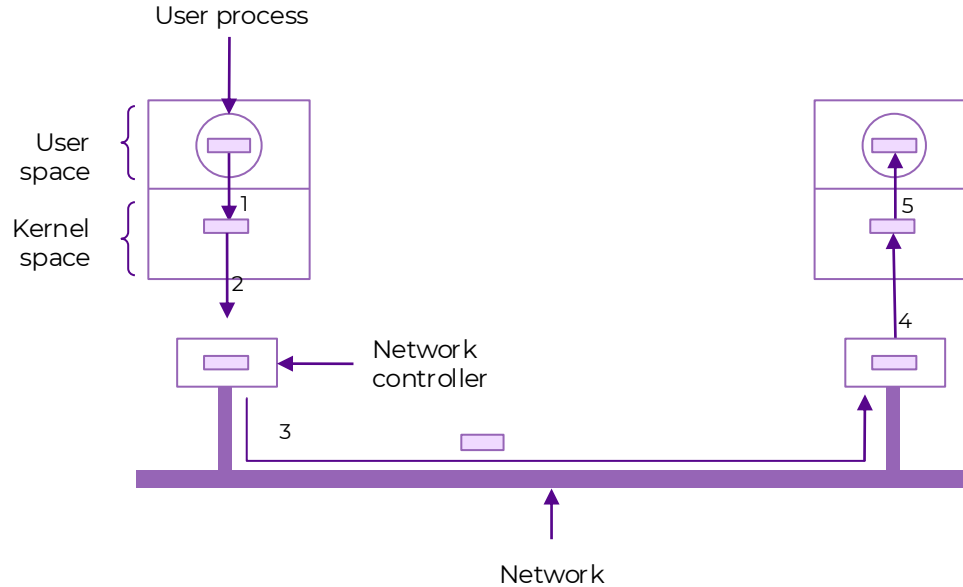


Figure 5-16. Networking may involve many copies of a packet.

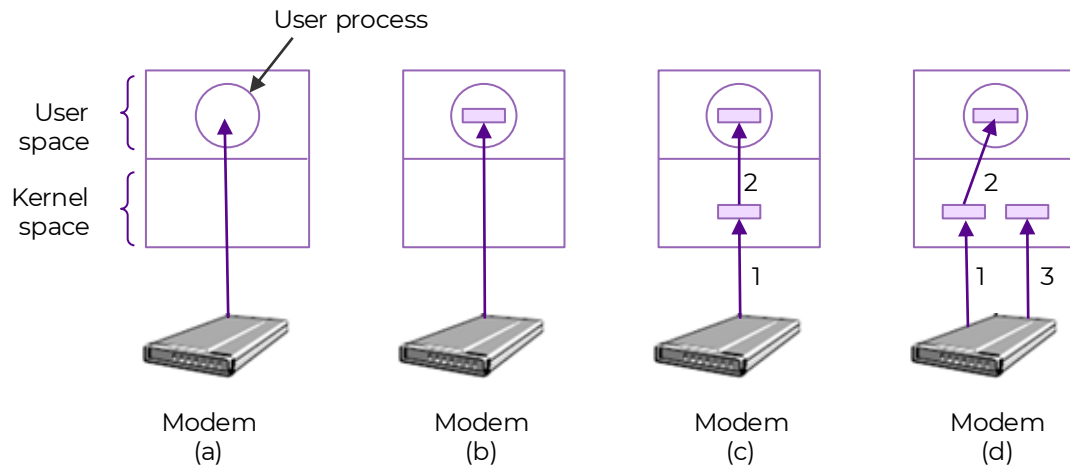
Buffering

- For efficiency, we don't want to give data back to a process in small pieces
- So we should store it up in some buffer until enough has accumulated

Buffering

- Where do we store the buffer?
 - In user space?
No – might have to swap out user page, but I/O has to go somewhere
 - In kernel space?
Better – but now what happens when the kernel needs to copy things to the user, but data is still coming in?
- In kernel space with *two buffers* – double buffering

Buffering



- a. Unbuffered input.
- b. Buffering in user space.
- c. Buffering in the kernel followed by copying to user space.
- d. Double buffering in the kernel.

Circular / Ring Buffer

- A popular data structure for buffering I/O is the *circular (or ring) buffer*
- Keep a buffer and two pointers
 - One to the first free slot in the buffer; can write there and advance the pointer
 - One to the first unprocessed item in the buffer; can read from there and advance the pointer
- When either pointer reaches the end of the buffer, wrap around to the start

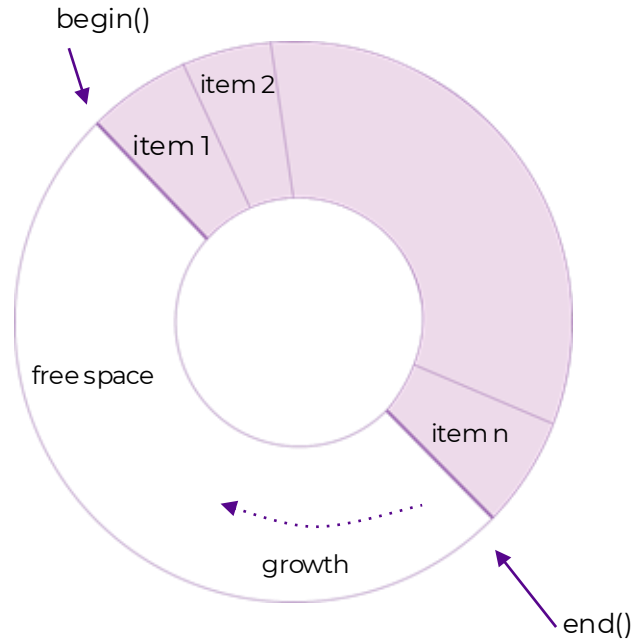


Image by Jan Gaspar, The Boost Book

```

    if (groupinfo->blocks[0] != groupinfo->small_
        int i;
        if (groupinfo->blocks[0] != groupinfo->small_
            for (i = 0; i < groupinfo->nblocks; i++)
                int i;
                freepage((unsigned long)groupinfo->blo
                for (i = 0; i < groupinfo->nblocks; i++)
                    }
                    freepage((unsigned long)groupinfo->blo
                    kfree(groupinfo);
                    }
                    kfree(groupinfo);
                }
            }
        }
    }
    EXPORT_SYMBOL(groupsfree);
    EXPORT_SYMBOL(groupsfree);
    /* export the groupinfo to a user-space array */
    /* export the groupinfo to a user-space array */
    const struct group_info *group_info)
    static int groups_touser(gid_t_user *groupList,
        {
            const struct group_info *group_info)
            int i;
            {
                unsigned int count = groupinfo->ngroups;
                int i;
                unsigned int count = groupinfo->ngroups;
                for (i = 0; i < groupinfo->nblocks; i++) {
                    unsigned int cpcount = min(NGROUPSPERBLOCK
                    for (i = 0; i < groupinfo->nblocks; i++) {
                        unsigned int len = cpcount * sizeof(*group
                        unsigned int cpcount = min(NGROUPSPERBLOCK
                        cpcount * sizeof(*group

```

- I/O Hardware
- I/O Techniques
- I/O Software
- OS I/O Design Issues



“

Keyboard

—

Keyboards

- Each keypress generates an interrupt
- Information about which key it was can be read out using port I/O

xv6 Keyboard Init

```
void
consoleinit(void)
{
    initlock(&cons.lock, "console");
    initlock(&input.lock, "input");

    devsw[CONSOLE].write = consolewrite;
    devsw[CONSOLE].read = consoleread;
    cons.locking = 1;

    picenable(IRQ_KBD);
    ioapicenable(IRQ_KBD, 0);
}
```

Reading One Character

```
int  
kbdgetc(void)  
{  
    static uint shift;  
    static uchar *charcode[4] = {  
        normalmap, shiftmap, ctlmap, ctlmap  
    };  
    uint st, data, c;  
  
    st = inb(KBSTATP);  
    if((st & KBS_DIB) == 0)  
        return -1;  
    data = inb(KBDATAP);  
  
    if(data == 0xE0){  
        shift |= E0ESC;  
        return 0;  
    } else if(data & 0x80){  
        // Key released  
        data = (shift & E0ESC ? data : data &  
0x7F);  
        shift &= ~(shiftcode[data] | E0ESC);  
        return 0;  
    } else if(shift & E0ESC){  
        // Last character was an E0 escape; or with 0x80  
        data |= 0x80;  
        shift &= ~E0ESC;  
    }  
}
```

```
    shift |= shiftcode[data];  
    shift ^= togglecode[data];  
    c = charcode[shift & (CTL | SHIFT)][data];  
    if(shift & CAPSLOCK){  
        if('a' <= c && c <= 'z')  
            c += 'A' - 'a';  
        else if('A' <= c && c <= 'Z')  
            c += 'a' - 'A';  
    }  
    return c;  
}
```

Console Interrupt

```
void
kbdintr(void)
{
    consoleintr(kbdgetc);
}
void
trap(struct trapframe *tf)
{
    ...
    case T_IRQ0 + IRQ_KBD:
        kbdintr();
        lapiceoi();
        break;
    ...
}
```



```
void
consoleintr(int (*getc)(void))
{
    int c, doprocdump = 0;
    acquire(&cons.lock);
    while((c = getc()) >= 0){
        switch(c){
            case C('P'): // Process listing.
                doprocdump = 1; // procdump() locks cons.lock indirectly;
                // invoke later
                break;
            case C('U'): // Kill line.
                while(input.e != input.w &&
                    input.buf[(input.e-1) % INPUT_BUF] != '\n'){
                    input.e--;
                    consputc(BACKSPACE);
                }
                break;
            case C('H'): case '\x7f': // Backspace
                if(input.e != input.w){
                    input.e--;
                    consputc(BACKSPACE);
                }
                break;
            default:
                if(c != 0 && input.e-input.r < INPUT_BUF){
                    c = (c == '\r') ? '\n' : c;
                    input.buf[input.e++ % INPUT_BUF] = c;
                    consputc(c);
                    if(c == '\n' || c == C('D') || input.e ==
                        input.r+INPUT_BUF){
                        input.w = input.e;
                        wakeup(&input.r);
                    }
                }
                break;
        }
    }
    release(&cons.lock);
    if(doprocdump) {
        procdump(); // now call procdump() wo. cons.lock held
    }
}
```

Keyboard Interrupt Handling

- Keys are buffered until a full line is entered in a circular buffer
- When the user presses enter, we wake up any process waiting for keyboard input (saw this previously)

```
#define INPUT_BUF 128
struct {
    struct spinlock lock;
    char buf[INPUT_BUF];
    uint r; // Read index
    uint w; // Write index
    uint e; // Edit index
} input;
```

An aerial photograph of Central Park in New York City, showing the Bethesda Fountain in the foreground and the Midtown skyline in the background. The word "Timers" is overlaid in large white letters, with a purple quotation mark above it. A purple horizontal line is positioned below the fountain.

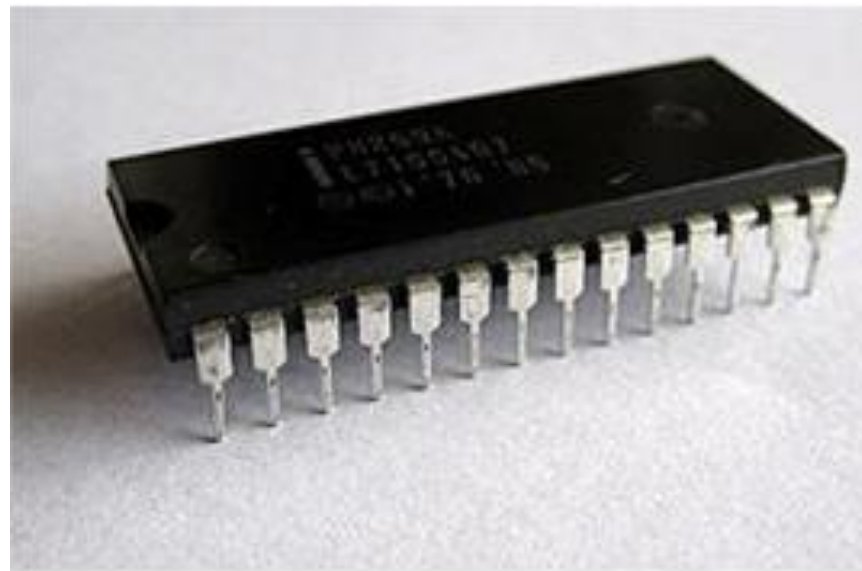
“ Timers

Timers

- Timers keep the OS running – we have seen how they allow preemptive scheduling to work
- Generally, implemented in hardware using a quartz crystal oscillator
- Can be programmed for *periodic* interrupts or *one-shot* (fire once and then disable)

xv6 Timer

- In uniprocessor mode: xv6 uses the Intel 8253/8254/82C54 Programmable Interval Timer (PIT)
- In SMP mode: APIC timer



Timer Interrupt Handling

```
void
trap(struct trapframe *tf)
{
[...]
```

```
    switch(tf->trapno) {
        case T_IRQ0 + IRQ_TIMER:
            if(cpu->id == 0){
                acquire(&tickslock);
                ticks++;
                wakeup(&ticks);
                release(&tickslock);
            }
            lapiceoi();

            break;
[...]
```

```
        // Force process to give up CPU on clock tick.
        // If interrupts were on while locks held, would need to check nlock.
        if(proc && proc->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER)
            yield();
[...]
```

```
    }
```

Class 18: When you type on the keyboard, when should the OS wake up a program waiting for input?

On every keypress

0%

Only when Enter is pressed

0%

When the buffer is full

0%

When a full line is ready or the buffer is full

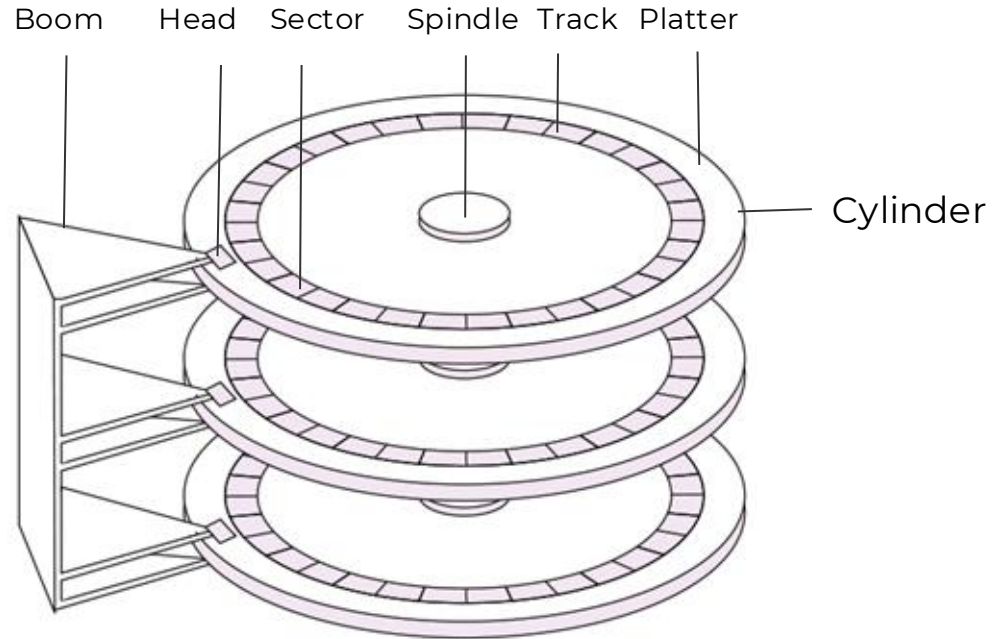
0%



Disk drive

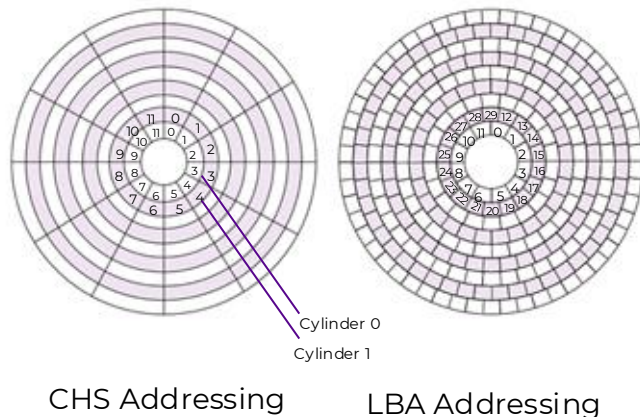
Disks

- Basic terminology:
 - Head
 - Track
 - Cylinder
 - Sector



Disks - Addressing

- Cylinder/Head/Sector is cumbersome to use
- Easier to just have a linear addressing scheme –this is called Logical Block Addressing (LBA)



Disk Protocols

- **IDE** – introduced 1986, max transfer 100MB/s (original drives – ~16MB/s)
- **SCSI** – introduced in 1981, still in use today
- **SATA** – introduced 2003, major consumer standard in use now, up to 16 Gb/s

xv6 IDE Driver

- Implemented in ide.c
- Implements an Interrupt-Driven I/O IDE driver (no DMA)
- Maintains a queue of disk blocks that need to be read or written
- A higher layer, in bio.c, handles caching blocks in memory using LRU algorithm

Sectors == buffers

Each buffer represents the content of one disk sector on a particular disk device.

```
1 struct buf {
2     int flags;
3     uint dev;
4     uint blockno;
5     struct buf *prev; // LRU cache List
6     struct buf *next;
7     struct buf *qnext; // disk queue
8     uchar data[BSIZE];
9 };
10 #define B_BUSY 0x1 // buffer is Locked by some process
11 #define B_VALID 0x2 // buffer has been read from disk
12 #define B_DIRTY 0x4 // buffer needs to be written to disk
```

IDE Initialization

- Talk to interrupt controller to enable interrupts for the IDE controller
- Wait for the drive to become ready
- Poll for a second hard drive
- Set the current drive to the first disk

```
void
ideinit(void)
{
    int i;
    initlock(&idelock, "ide");
    picenable(IRQ_IDE);
    ioapicenable(IRQ_IDE, ncpu - 1);
    idewait(0);
    // Check if disk 1 is present
    outb(0x1f6, 0xe0 | (1<<4));
    for(i=0; i<1000; i++){
        if(inb(0x1f7) != 0){
            havedisk1 = 1;
            break;
        }
    }
    // Switch back to disk 0.
    outb(0x1f6, 0xe0 | (0<<4));
}
```

Sending Data

Add buffer b to the end of the queue

Start disk if needed

Wait for finish, by sleeping

```
4250 // Sync buf with disk.
4251 // If B_DIRTY is set, write buf to disk, clear B_DIRTY, set B_VALID.
4252 // Else if B_VALID is not set, read buf from disk, set B_VALID.
4253 void
4254 iderw(struct buf *b)
4255 {
4256     struct buf **pp;
4257
4258     if (! (b->flags & B_BUSY))
4259         panic ("iderw: buf not busy");
4260     if ((b->flags & (B_VALID | B_DIRTY)) == B_VALID)
4261         panic ("iderw: nothing to do");
4262     if (b->dev != 0 && !havedisk1)
4263         panic ("iderw: ide disk 1 not present");
4264
4265     acquire (&idelock);
4266
4267     // Append b to idequeue.
4268     b->qnext = 0;
4269     for (pp=&idequeue; *pp; pp=(*pp) ->qnext)
4270         ;
4271     *pp = b;
4272
4273     // Start disk if necessary.
4274     if (idequeue == b)
4275         idestart (b);
4276
4277     // Wait for request to finish.
4278     while ((b->flags & (B_VALID | B_DIRTY)) != B_VALID) {
4279         sleep (b, &idelock);
4280     }
4281
4282     release (&idelock);
4283 }
```

Sending Commands

Data written using out instruction

```
// Start the request for b. Caller must hold idelock.
static void
idestart(struct buf *b)
{
    if(b == 0)
        panic("idestart");
    if(b->blockno >= FSSIZE)
        panic("incorrect blockno");
    int sector_per_block = BSIZE/SECTOR_SIZE;
    int sector = b->blockno * sector_per_block;
    if (sector_per_block > 7) panic("idestart");
    idewait(0);
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, sector_per_block); // number of sectors
    outb(0x1f3, sector & 0xff);
    outb(0x1f4, (sector >> 8) & 0xff);
    outb(0x1f5, (sector >> 16) & 0xff);
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((sector>>24)&0x0f));
    if(b->flags & B_DIRTY){
        outb(0x1f7, IDE_CMD_WRITE);
        outsl(0x1f0, b->data, BSIZE/4);
    } else {
        outb(0x1f7, IDE_CMD_READ);
    }
}
```

Issues a read or write depending on flags

Interrupt Handling

Call main IDE interrupt handler

Acknowledge
interrupt complete

```
void
trap(struct trapframe *tf)
{
    [...]
    case T_IRQ0 + IRQ_IDE:
        ideintr();
        lapiceoi();
        break;
    [...]
}
```

Interrupt Handling

Data actually read using instruction

Inside each interrupt we kick off the next read/write

```
// Interrupt handler.
void
ideintr(void)
{
    struct buf *b;
    // First queued buffer is the active request.
    acquire(&idelock);
    if((b = idequeue) == 0){
        release(&idelock);
        // cprintf("spurious IDE interrupt\n");
        return;
    }
    idequeue = b->qnext;
    // Read data if needed.
    if(!(b->flags & B_DIRTY) && idewait(1) >= 0)
        insl(0x1f0, b->data, BSIZE/4);
    // Wake process waiting for this buf.
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b);
    // Start disk on next buf in queue.
    if(idequeue != 0)
        idestart(idequeue);
    release(&idelock);
}
```

Still Plenty More Hardware

- To have a functional OS, we still need more hardware support:
 - **Graphics** (at least text console graphics) – `cgaputc()` in `console.c`
 - Serial port – `uart.c`
 - The other parts of the APIC (to route interrupts to the correct CPU) – `mp.c`, `lapic.c`, `ioapic.c`
- If you're curious, feel free to read up on how they're implemented

Appendix